

Pre-Processing

- Dropping null answers rows
- Lowercase entire data frame
- Dropping duplicated rows in 4 cols
- Remove extra spaces / line breaks on question and answer column
- Remove special characters / punctuation noise (if not medically relevant) on question column
- Remove (are) in question column
- Check spelling mistakes
- Group by on unique questions and join all answers of same questions
- Dropped scope and focus area column
- Reduce dataset from 16359, 4 to 14322, 2
- Used groq api script to shorten the answers and remove garbage like emails, URLs, and unrelated material. (only 1000 rows)
- Clean the short answer column
- replace the short answer column with long answer column
- Used 1k rows for finetuning.

Fine-Tuning

- Model Options Explored
 - BERT-family (BioBERT, PubMedBERT): Strong retrievers, not generators.
 - BioGPT: Biomedical generative model, but tends to output technical/jargony text.
 - Large LLMs (LLaMA, Falcon, GPT-3.5/4, Med-PaLM): Too large, closed-source, or impractical for small dataset fine-tuning.
 - Flan-T5: Instruction-tuned, encoder-decoder, natural QA style, strong for small/medium data.
- Decision Criteria for Model Selection
 - Task alignment: Model should already handle QA (Flan-T5 > BioBERT).
 - Dataset size: 1k rows is small → avoid very large LLMs (risk overfitting).
 - Domain: Need balance between medical knowledge + user-friendly output.
 - Practicality: Open-source, easy to fine-tune (PEFT/LoRA).
- Final Model Choice
 - Best fit: google/flan-t5-base.
 - Reasons:
 - Instruction-tuned → already good at QA.
 - Small enough for 1k rows without overfitting.
 - Supports LoRA (parameter-efficient fine-tuning).
 - generates natural, user-friendly answers.

RAG pipeline

- Data Ingestion: Load the MedQuAD dataset (CSV format) into Colab.
- Text Preprocessing: Clean and structure the text (remove noise, keep questions/answers).
- Embedding Generation: Convert each question/answer into dense vector embeddings using a pretrained model (e.g., all-MiniLM-L6-v2).
- Vector Database Storage: Store embeddings in FAISS (or Pinecone/ChromaDB) for fast similarity search.
- Retriever: On a new user query, generate its embedding and retrieve the most relevant MedQuAD entries from the vector DB.
- Generator: Pass the retrieved context + user query to a fine-tuned LLM
- Answer Construction: The LLM generates a well-formed, context-aware answer using both the query and retrieved context.
- User Interaction: Return the answer to the client in natural language, with the option to show the supporting context.

This is the end-to-end Retrieval-Augmented Generation pipeline:

Dataset → Embeddings → Vector DB → Retriever → Generator → Final Answer.